

1. System Setup

Executed by: Central Bank CB and maintainer set $\mathbb{M}$ ($D = 5t + 1$)
Output: System public parameters
1 CB generates signature key pair (sk_{CB}, pk_{CB}) .
2 Maintainers jointly set up threshold ElGamal encryption: aggregate public key pk_{ThEnc} , each M_j obtains decryption share $sk_{ThEnc,j}$.
3 Set global parameters: holding limit H , per-transaction limit $V_{\max}$, commitment Comm , Groth16 curve, Schnorr group.

2. User Enrollment

Input: user's real identity (KYC), challenge c , epoch e , limit H
Output: secrets (a_U, sk_U) , initial scm_0 , signature σ_0 , share commitments $\{\text{Com}_j\}$
1 Choose $a_U, sk_U \in \mathbb{Z}_p$, set $pk_U = g^{sk_U}$.
2 Choose r_0 , compute $\text{scm}_0 \leftarrow \text{Comm}_{r_0}(0, 0, a_U, pk_U, H, e, 0, c)$. $\triangleright \text{bal}=0, \text{ctr}=0, \text{scmprev}=0, \text{ccm}=c$
3 Generate π_{enroll} ; send $(\pi_{\text{enroll}}, \text{scm}_0, pk_U, e, H, c)$ to CB .
4 Perform (t, D) -sharing of a_U : shares s_j , commitments $\text{Com}_j = g^{s_j} h^{r'_j}$, proof π_{share} ; broadcast.
5 CB verifies, returns σ_0 ; user stores $(\sigma_0, r_0, a_U, sk_U)$.

3. Offline Spend

3.1 RequestPayment (Receiver R)

Input: $\text{scm}_{R,\text{old}}$, amount v
Output: rcm
1 Choose r_{req} , compute $\text{rcm} = \text{Comm}_{r_{\text{req}}}(\text{scm}_{R,\text{old}})$.
2 Send (rcm, v) to sender S .

3.2 CreatePayment (Sender S)

Input: $(\text{scm}_{S,\text{old}}, \sigma_{S,\text{old}})$ with opening $(a_S, sk_S, \text{bal}_{\text{old}}, \text{ctr}_{\text{old}}, r_{\text{old}})$; receiver's rcm, v, pk_R ; dependency history hist_{rel} if needed.
Output: $\text{scm}_{S,\text{new}}, T_S, \psi_S, \text{dcm}_{S,\text{new}}$, proofs $\pi_{\text{state}}^{\text{create}}, \pi_{\text{pm}}, \pi_{\text{link}}, \pi_{\text{dep}}^{\text{create}}$.
1 Verify $0 < v \leq \text{bal}_{\text{old}}$; $\text{bal}_{\text{new}} = \text{bal}_{\text{old}} - v$, $\text{ctr}_{\text{new}} = \text{ctr}_{\text{old}} + 1$.
2 Choose new blindings $r_{\text{new}}, r_{\text{dep}}, k_S, \text{blind}_{\text{pm}}$.
3 $\text{scm}_{S,\text{new}} = \text{Comm}_{r_{\text{new}}}(\text{bal}_{\text{new}}, \text{ctr}_{\text{new}}, a_S, pk_S, H, e_S, \text{scm}_{S,\text{old}}, \text{rcm})$.
4 $\text{dcm}_{S,\text{new}} = \text{Comm}_{r_{\text{dep}}}(\text{scm}_{S,\text{old}}, \text{rcm})$.
5 $\text{pcm} = \text{Comm}_{\text{blind}_{\text{pm}}}(v, \text{rcm}, \text{scm}_{S,\text{new}}, e_S)$. $\triangleright$ compute locally; not sent
6 $T_S = g^{a_S \cdot \text{ctr}_{\text{new}}}$.
7 $\psi_S = (g^{k_S}, g^v \cdot pk_{ThEnc}^{k_S}, g^{pk_S} \cdot pk_{ThEnc}^{k_S}, g^{pk_R} \cdot pk_{ThEnc}^{k_S})$.
8 Generate in-circuit $\pi_{\text{state}}^{\text{create}}$ and π_{pm} ; generate out-of-circuit Schnorr π_{link} binding T_S, ψ_S to $\text{pcm}, \text{scm}_{S,\text{new}}$.
9 If $\text{scm}_{S,\text{old}}$ and dependencies are signed, generate $\pi_{\text{dep}}^{\text{create}}$; else mark missing.
10 Assemble hist_{rel} recursively; payment creation entries include $T, \psi, \pi_{\text{link}}$.
11 Send $(\text{blind}_{\text{pm}}, \text{scm}_{S,\text{new}}, \text{dcm}_{S,\text{new}}, T_S, \psi_S, \pi_{\text{state}}^{\text{create}}, \pi_{\text{pm}}, \pi_{\text{link}}, \text{hist}_{\text{rel}}, e_S)$ to receiver.

3.3 CompletePayment (Receiver R)

Input: $(\text{scm}_{R,\text{old}}, \sigma_{R,\text{old}})$ with opening $(a_R, sk_R, \text{bal}_{\text{old}}, \text{ctr}_{\text{old}}, r_{\text{old}})$; from S : $\text{blind}_{\text{pm}}, \text{scm}_{S,\text{new}}, T_S, \psi_S, \pi_{\text{state}}^{\text{create}}, \pi_{\text{pm}}, \pi_{\text{link}}, \text{hist}_{\text{rel}}, e_S$; amount v, pk_S .
Output: $\text{scm}_{R,\text{new}}, \text{dcm}_{R,\text{new}}$, proofs $\pi_{\text{state}}^{\text{complete}}$ (and possibly $\pi_{\text{dep}}^{\text{complete}}$).
1 Recompute $\text{pcm} = \text{Comm}_{\text{blind}_{\text{pm}}}(v, \text{rcm}, \text{scm}_{S,\text{new}}, e_S)$.
2 Verify $\pi_{\text{state}}^{\text{create}}, \pi_{\text{pm}}$ (with recomputed pcm), π_{link} , and all proofs in hist_{rel} .
3 $\text{bal}_{\text{new}} = \text{bal}_{\text{old}} + v$.
4 Choose $r_{\text{new}}, r_{\text{dep}}$.
5 $\text{scm}_{R,\text{new}} = \text{Comm}_{r_{\text{new}}}(\text{bal}_{\text{new}}, \text{ctr}_{\text{old}}, a_R, pk_R, H, e_R, \text{scm}_{R,\text{old}}, \text{scm}_{S,\text{new}})$.
6 $\text{dcm}_{R,\text{new}} = \text{Comm}_{r_{\text{dep}}}(\text{scm}_{R,\text{old}}, \text{scm}_{S,\text{new}})$.
7 Generate $\pi_{\text{state}}^{\text{complete}}$ (includes $\text{bal}_{\text{new}} \leq H, |e_S - e_R| \leq \Delta_{\text{sync}}$).
8 If $\text{scm}_{R,\text{old}}$ and $\text{scm}_{S,\text{new}}$ are signed, generate $\pi_{\text{dep}}^{\text{complete}}$; else mark missing.
9 Store own state record and merged history with sender's T_S, ψ_S .

4. Reconnect and Settle

Input (User): all unsigned state records in dependency order
Output: CB signatures (or double-spend rejection)
User side:
1 **for** each unsigned state in order **do**
2 **if** dependencies now signed **then**
3 generate corresponding π_{dep}
4 **end if**
5 create signature request (payment creation includes T, ψ ; completion only state+proofs)
6 upon receiving signature, store it
7 **end for**
Bank side:
8 Verify $\pi_{\text{state}}, \pi_{\text{dep}}, \pi_{\text{pm}}, \pi_{\text{link}}$ as applicable.
9 **if** payment creation **then**
10 **if** T already in ledger with $\text{scm}' \neq \text{scm}$ **then**
11 double-spend; reject
12 **else**
13 record T
14 **end if**
15 **end if**
16 Sign scm , write $(\text{scm}, \sigma, T, \psi)$ into ledger (completion: scm, σ only).
17 Return signature.

5. Audit (Privacy Revocation)

Input: ciphertext ψ , decryption shares
Output: (v, pk_S, pk_R) and identities
1 At least $t + 1$ maintainers compute $d_j = \psi \cdot c_1^{sk_{ThEnc,j}}$ with Schnorr proof.
2 Collect $\beta = t + 1$ valid shares
3 Recover: $g^v = \psi \cdot c_2 / \prod d_j^{\lambda_j}, g^{pk_S} = \psi \cdot c_3 / \prod d_j^{\lambda_j}, g^{pk_R} = \psi \cdot c_4 / \prod d_j^{\lambda_j}$.
4 Brute-force v ($\leq V_{\max}$); look up pk_S, pk_R in directory.

6. Trace (User's Entire History)

Input: user U 's identifier or share commitments $\{\text{Com}_j\}$
Output: list of (T, role)
1 Retrieve $\{\text{Com}_j\}$ from registration.
2 Threshold-compute g^{a_U} : gather $E_j = g^{s_j}$ with proofs, $g^{a_U} = \prod E_j^{\lambda_j}$.
3 Sender trail:
4 $T_1 = g^{a_U}$; for $\text{ctr} = 1, 2, \dots$:
5 **if** T_{ctr} in ledger, record as sender; $T_{\text{ctr}+1} = T_{\text{ctr}} \cdot g^{a_U}$.
6 Receiver trail:
7 For each ψ in ledger, decrypt pk_R ; if $pk_R = pk_U$, record as receiver.
8 Merge and output.

Enrollment π_{enroll}

- Public: $pk_{CB}, scm_0, pk_U, e, c, H$
- Secret: a_U, sk_U, r_0
 - $scm_0 = \text{Comm}_{r_0}(0, 0, a_U, pk_U, H, e, 0, c)$
 - $pk_U = g^{sk_U}$

Payment creation dependency $\pi_{\text{dep}}^{\text{create}}$

- Public: $pk_{CB}, dcm_{S,\text{new}}$
- Secret: $r_{\text{dep}}, scm_{S,\text{old}}, rcm, \sigma$
 - $dcm_{S,\text{new}} = \text{Comm}_{r_{\text{dep}}}(scm_{S,\text{old}}, rcm)$
 - $\text{Ver}(pk_{CB}, scm_{S,\text{old}}, \sigma) = \text{True}$

Payment proof π_{pm}

- Public: pcm
- Secret: $v, rcm, scm_{S,\text{new}}, e_S, \text{blind}_{\text{pm}}$
 - $pcm = \text{Comm}_{\text{blind}_{\text{pm}}}(v, rcm, scm_{S,\text{new}}, e_S)$

Payment creation state $\pi_{\text{state}}^{\text{create}}$

- Public: $dcm_{S,\text{new}}, dcm_{R,\text{new}}, pcm, pk_{CB}$
- Secret: $a_S, pk_S, bal, ctr, e_S, v, r_{\text{old}}, r_{\text{new}}, \sigma, rcm, r_{\text{dep}}, \text{blind}_{\text{pm}}$
 - $scm_{S,\text{old}} = \text{Comm}_{r_{\text{old}}}(bal, ctr, a_S, pk_S, H, e_S, scm_{S,\text{prev}}, ccm_{S,\text{old}})$
 - $\text{Ver}(pk_{CB}, scm_{S,\text{old}}, \sigma) = \text{True}$
 - $scm_{S,\text{new}} = \text{Comm}_{r_{\text{new}}}(bal - v, ctr + 1, a_S, pk_S, H, e_S, scm_{S,\text{old}}, rcm)$
 - $dcm_{S,\text{new}} = \text{Comm}_{r_{\text{dep}}}(scm_{S,\text{old}}, rcm)$
 - $pcm = \text{Comm}_{\text{blind}_{\text{pm}}}(v, rcm, scm_{S,\text{new}}, e_S)$
 - $bal \geq v, \quad bal - v \leq H$

Payment completion dependency $\pi_{\text{dep}}^{\text{complete}}$

- Public: $pk_{CB}, dcm_{R,\text{new}}$
- Secret: $r_{\text{dep}}, scm_{R,\text{old}}, scm_{S,\text{new}}, \sigma_R, \sigma_S$
 - $dcm_{R,\text{new}} = \text{Comm}_{r_{\text{dep}}}(scm_{R,\text{old}}, scm_{S,\text{new}})$
 - $\text{Ver}(pk_{CB}, scm_{R,\text{old}}, \sigma_R) = \text{True}$
 - $\text{Ver}(pk_{CB}, scm_{S,\text{new}}, \sigma_S) = \text{True}$

Payment completion state $\pi_{\text{state}}^{\text{complete}}$

- Public: $scm_{R,\text{new}}, dcm_{R,\text{new}}, pcm, pk_{CB}$
- Secret: $a_R, pk_R, bal, ctr, e_R, v, r_{\text{old}}, r_{\text{new}}, \sigma, rcm, \text{blind}_{\text{pm}}, e_S, scm_{S,\text{new}}$
 - $scm_{R,\text{old}} = \text{Comm}_{r_{\text{old}}}(bal, ctr, a_R, pk_R, H, e_R, scm_{R,\text{prev}}, ccm_{R,\text{old}})$
 - $\text{Ver}(pk_{CB}, scm_{R,\text{old}}, \sigma) = \text{True}$
 - $scm_{R,\text{new}} = \text{Comm}_{r_{\text{new}}}(bal + v, ctr, a_R, pk_R, H, e_R, scm_{R,\text{old}}, scm_{S,\text{new}})$
 - $dcm_{R,\text{new}} = \text{Comm}_{r_{\text{dep}}}(scm_{R,\text{old}}, scm_{S,\text{new}})$
 - $pcm = \text{Comm}_{\text{blind}_{\text{pm}}}(v, rcm, scm_{S,\text{new}}, e_S)$
 - $bal + v \leq H$
 - $|e_S - e_R| \leq \Delta_{\text{sync}}$

Link proof π_{link} (Schnorr)

- Public: $pcm, scm_{S,\text{new}}, T_S, \psi_S, pk_{ThEnc}, g$
- Secret: $a_S, ctr, v, pk_S, pk_R, k_S, rcm, \text{blind}_{\text{pm}}, r_{\text{new}}, bal_{\text{new}}, e_S, H, scm_{S,\text{old}}$
 - $pcm = \text{Comm}_{\text{blind}_{\text{pm}}}(v, rcm, scm_{S,\text{new}}, e_S)$
 - $scm_{S,\text{new}} = \text{Comm}_{r_{\text{new}}}(bal_{\text{new}}, ctr, a_S, pk_S, H, e_S, scm_{S,\text{old}}, rcm)$
 - $T_S = g^{a_S \cdot \text{ctr}}$
 - $\psi_S = (g^{k_S}, g^v pk_{ThEnc}^{k_S}, g^{pk_S} pk_{ThEnc}^{k_S}, g^{pk_R} pk_{ThEnc}^{k_S})$

User Wallet (Registration):

- Secrets: (a_U, sk_U, r_0) , Initial State: scm_0
- Signature: σ_0 , Share commitments: $\{\text{Com}_j\}$
- State Commitment: $scm_0 = \text{Comm}_{r_0}(0, 0, a_U, pk_U, H, e, 0, c)$.

System Public Parameters:

- Keys: pk_{CB}, pk_{ThEnc} , Threshold: (t, D)
- Constraints: Limit H , Max value V_{max} , Sync tolerance Δ_{sync}
- Primitives: Commitment Comm , Groth16, Schnorr group

Current Offline States:

- Alice (Sender): $(scm_{S,\text{old}}, \sigma_S, \text{old})$
- Bob (Receiver): $(scm_{R,\text{old}}, \sigma_R, \text{old})$



Alice (Sender)



Bob (Receiver)

2. Payment Request: (rcm, v)

4. Payment Creation:

 $\text{blind}_{\text{pm}}, scm_{S,\text{new}}, dcm_{S,\text{new}}, T_S, \psi_S, e_S, \pi_{\text{state}}^{\text{create}}, \pi_{\text{pm}}, \pi_{\text{link}}, \text{hist}_{\text{rel}}$ **3. CreatePayment (Sender)**

- **Update State:** $bal_{\text{new}} = bal_{\text{old}} - v; \text{ctr}_{\text{new}} = \text{ctr}_{\text{old}} + 1$
- **Commitments:**
 - $scm_{S,\text{new}} = \text{Comm}_{r_{\text{new}}}(bal_{\text{new}}, \text{ctr}_{\text{new}}, a_S, pk_S, H, e_S, scm_{S,\text{old}}, rcm)$
 - $dcm_{S,\text{new}} = \text{Comm}_{r_{\text{dep}}}(scm_{S,\text{old}}, rcm)$
 - $pcm = \text{Comm}_{\text{blind}_{\text{pm}}}(v, rcm, scm_{S,\text{new}}, e_S)$
- **Traceability & Audit:**
 - Double-spend tag: $T_S = g^{a_S \cdot \text{ctr}_{\text{new}}}$
 - Audit ciphertext: $\psi_S = (g^{k_S}, g^v pk_{ThEnc}^{k_S}, g^{pk_S} pk_{ThEnc}^{k_S}, g^{pk_R} pk_{ThEnc}^{k_S})$
- **Generate Proofs:** $\pi_{\text{state}}^{\text{create}}, \pi_{\text{pm}}; \pi_{\text{link}}(\text{link } T_S, \psi_S); \pi_{\text{dep}}^{\text{create}}$ if old state signed.
- **Assemble history hist_{rel} :**
 - For the new payment creation state $scm_{S,\text{new}}$, include its record $(scm_{S,\text{new}}, dcm_{S,\text{new}}, T_S, \psi_S, \pi_{\text{state}}^{\text{create}}, \pi_{\text{pm}}, \pi_{\text{link}})$.
 - For each unsigned dependency of $scm_{S,\text{new}}$ (starting from $scm_{S,\text{old}}$ and recursively backwards):
 - If the dependency is a payment creation, include its record with $(T, \psi, \pi_{\text{link}})$.
 - If the dependency is a payment completion, include its record without $(T, \psi, \pi_{\text{link}})$.
 - Stop recursion at the first state that already has a valid bank signature σ in the local history.

1. RequestPayment (Recipient)

- Choose random blinding r_{req}
- Compute request commitment: $rcm = \text{Comm}_{r_{\text{req}}}(scm_{R,\text{old}})$

5. AcceptPayment (Recipient)

- **Recompute pcm**
- **Verify proofs:** Check π_{pm} with recomputed pcm; verify $\pi_{\text{state}}^{\text{create}}, \pi_{\text{link}}$.
- **Recursive dependency check:** Starting from $scm_{S,\text{new}}$, verify every state in hist_{rel} :
 - A valid signature σ exists for the state, **or**
 - The state provides valid $\pi_{\text{state}}, \pi_{\text{dep}}$ (and π_{pm} for completions), and dependencies pass recursively.
- **Epoch validity:** Ensure sender e_S is recent and $|e_S - e_R| \leq \Delta_{\text{sync}}$.

6. CompletePayment (Recipient)

- **Update State:**
 - $bal_{\text{new}} = bal_{\text{old}} + v$
 - $scm_{R,\text{new}} = \text{Comm}_{r_{\text{new}}}(bal_{\text{new}}, \text{ctr}_{\text{old}}, a_R, pk_R, H, e_R, scm_{R,\text{old}}, scm_{S,\text{new}})$
 - $dcm_{R,\text{new}} = \text{Comm}_{r_{\text{dep}}}(scm_{R,\text{old}}, scm_{S,\text{new}})$
- **Generate $\pi_{\text{state}}^{\text{complete}}$** (includes $bal_{\text{new}} \leq H, |e_S - e_R| \leq \Delta_{\text{sync}}$).
- If $scm_{R,\text{old}}$ and $scm_{S,\text{new}}$ are signed, generate $\pi_{\text{dep}}^{\text{complete}}$.
- **Save merged history** (including sender's T_S, ψ_S).